

Executor-Inclusive Inner Product Proofs for Validator-Side Deterministic Settlement on Solana - Draft

Blessed Tosin-Oyinbo (tnxl)

April 2025

1 Summary and Overview

This paper presents a cryptographic protocol and validator-runtime integration mechanism that allows execution results to be committed as algebraic proofs over vector commitments, using Pedersen Commitments and Inner Product Proofs (IPP). It is designed to align with Solana’s high-throughput and deterministic validator model, this approach enables executor modules to offload computation while allowing validators to verify correctness without re-execution or replaying transaction states. We provide a complete mathematical foundation, high-level integration sketch with Solana’s runtime and transaction format, and highlight how this can extend this cryptographic protocol for scalable, verifiable execution flows.

2 Introduction

Solana’s architecture prioritizes high throughput and deterministic state transitions. While this model scales well under uniform execution (e.g., Sealevel), it imposes limitations when validators must re-execute arbitrary or non-native computations. To address this, we propose a protocol enabling executor-side state transition proofs that are compact, algebraic, and verifiable by validators without requiring full re-execution.

The core idea is to commit to state vectors using Pedersen commitments and prove relations over them using recursive inner product arguments, extending the previously proposed SIMD-X protocol.

3 Background: Pedersen-Vector Commitments and Inner Product Proofs

Let $\vec{x}, \vec{w} \in \mathbb{Z}_q^n$ represent two secret vectors.

For this protocol, we define a **Pedersen-Vector Commitment** over independent generators $\{G_i\}, \{H_i\}, G_r \in \mathbb{G}$ as:

$$P = \sum_{i=1}^n x_i G_i + w_i H_i + r G_r$$

Here $r \in \mathbb{Z}_q$ is designated as a blinding scalar.

We aim to prove that

$$\langle \vec{x}, \vec{w} \rangle = c$$

...without revealing $\vec{x}, \vec{w}$ or r .

This can be achieved using a recursive **inner product proof** (IPP), similar to the Bulletproofs construction, producing a logarithmic-size transcript $(L_i, R_i), x_i$, and final scalars a, b, r' .

4 Solana Transaction Format Overview

Solana transactions contain:

- **Message:** A set of ordered instructions and account access declarations.
- **Signatures:** Ed25519 signatures to authorize transaction execution.
- **Accounts:** Read/write accounts that are preloaded for instruction execution.

Under the current model, validators deterministically re-execute all execution outputs from Specialized Execution Enviroments (SEEs). This ensures consistency but in the long run, burdens validators under modular or off-chain execution models.

5 Executor $\rightarrow$ Validator Proof Protocol

We now describe how to integrate inner product-based proofs into Solana:

5.1 Executor Steps

1. Compute output vectors $\vec{x}, \vec{w}$ from transaction logic.
2. Generate a Pedersen commitment P to the result.
3. Compute the scalar inner product $c = \langle \vec{x}, \vec{w} \rangle$.
4. Generate a recursive inner product proof transcript.
5. Submit transaction with $(P, c, \text{transcript}, a, b, r')$ as metadata.

5.2 Validator Steps

1. Parse metadata from the transaction.
2. Recompute transcript challenges via Fiat–Shamir hash of (L_i, R_i) .
3. Fold the commitment P into $P_{\text{compressed}}$ using transcript.
4. Verify:

$$P_{\text{compressed}} \stackrel{?}{=} aG + bH + r'G_r \quad \text{and} \quad c \stackrel{?}{=} ab$$

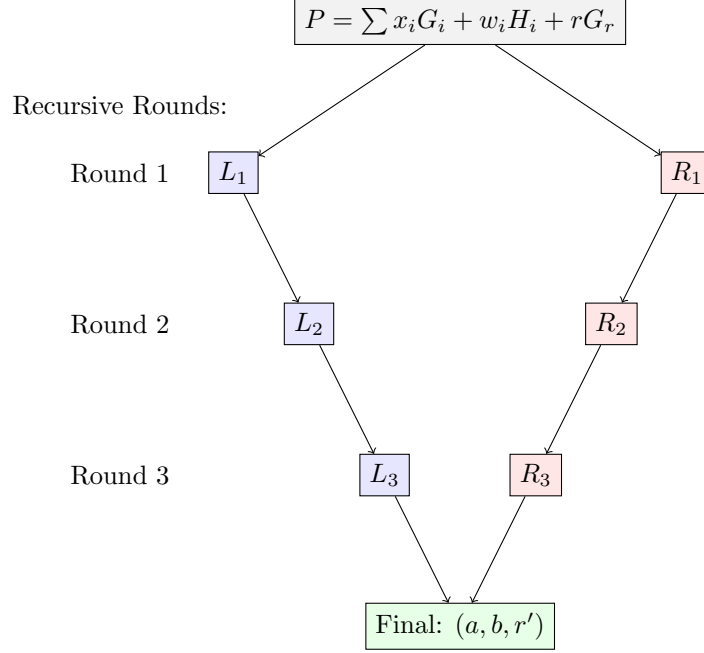


Figure 1: Recursive folding of the commitment using inner product proof transcript. Each round compresses the vectors by half.

6 Runtime Changes Required

Required Additions

- Extend transaction format to include proof metadata (commitment, scalar, transcript).
- Integrate scalar curve operations in validator runtime for stateless verification.
- Maintain temporal context binding (e.g., slot s) to avoid replay attacks.

Optional Optimizations

- Batch verification of SIMD proofs.
- SIMD-parallel EC arithmetic for transcript folding.
- Commit-to-account model where accounts attest to internal state transitions via IPPs.

7 Theoretical Properties

- **Perfect Hiding:** Pedersen commitments hide $\vec{x}, \vec{w}$ unconditionally.
- **Computational Binding:** Based on hardness of discrete log in $\mathbb{G}$.
- **Non-interactive:** Fiat–Shamir heuristic removes need for verifier interaction.
- **Logarithmic Proof Size:** $O(\log n)$ transcript; suitable for low-latency systems.

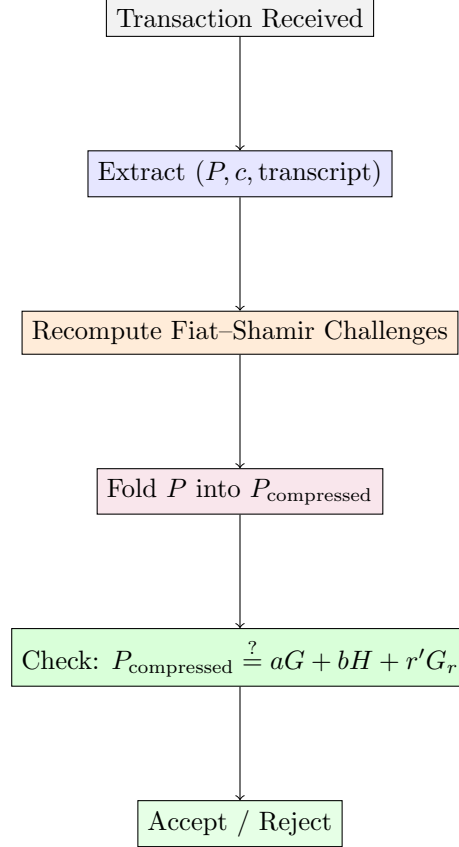


Figure 2: Validator-side verification pipeline for inner product proofs. Stateless EC algebra replaces full re-execution.

- **Parallelizable Verification:** Stateless algebra enables vectorized checks.
- **Compatibility:** Can be integrated into existing Solana runtime without consensus changes.

8 Comparison with POPK-Based

Compared to SIMD-X v1 (our previous approach), which is based on proof-of-partial-knowledge (POPK), the latest protocol (PVC & IPP) (v2) introduces several significant enhancements. While both versions support partial inclusion proofs, only v2 supports full linear relation proofs, thanks to its use of inner product arguments. The proof size in v1 remains constant, whereas v2 incurs a logarithmic overhead — a trade-off for increased expressiveness and cryptographic guarantees. In terms of privacy, v1 enables selective disclosure, while v2 offers full zero-knowledge privacy over both vectors. Although verifier load remains low in both models, v2 introduces algebraic operations (e.g., elliptic curve group computations). Finally, while the POPK protocol has limited compatibility with zero-knowledge systems, PVC IPP (v2) is fully ZK-compatible, making it suitable for integration into recursive proof systems and zkSNARK backends, and particularly suitable with Solana’s architecture if properly implemented.

9 Future Work

Future upgrades and iterations of this protocol would solely focus on deeper alignment with Solana’s architecture, making sure that it achieves seamless integration without imposing technical, economic, or crypto-economic overhead.

Key directions include designing native support for inner product proof generation from zkVM-based runtimes (e.g., WASM, MoveVM, or EVM rollup environments), enabling recursive IPP proofs for composable execution chains, and exploring zkSNARK compression techniques such as Groth16 and Plonk to practically reduce the proof sizes to a constant.

Overall a critical milestone is the development of a high-performance on-chain verifier implemented as a Solana-native crate or BPF program, enabling fast, stateless verification of proofs directly within the validator runtime without disrupting Solana’s ultra-low-latency pipeline.

10 Conclusion

The Pedersen-Vector commitment + IPP protocol extends the original proof-of-partial-knowledge design to support recursive inner product proofs over vector commitments. This would enable executors to offload arbitrary logic while allowing validators to verify correctness deterministically using only algebraic operations.

It combines cryptographic rigor with real-world applicability on fast, low-latency networks like Solana, unlocking new execution models for privacy, scalability, and interoperability.